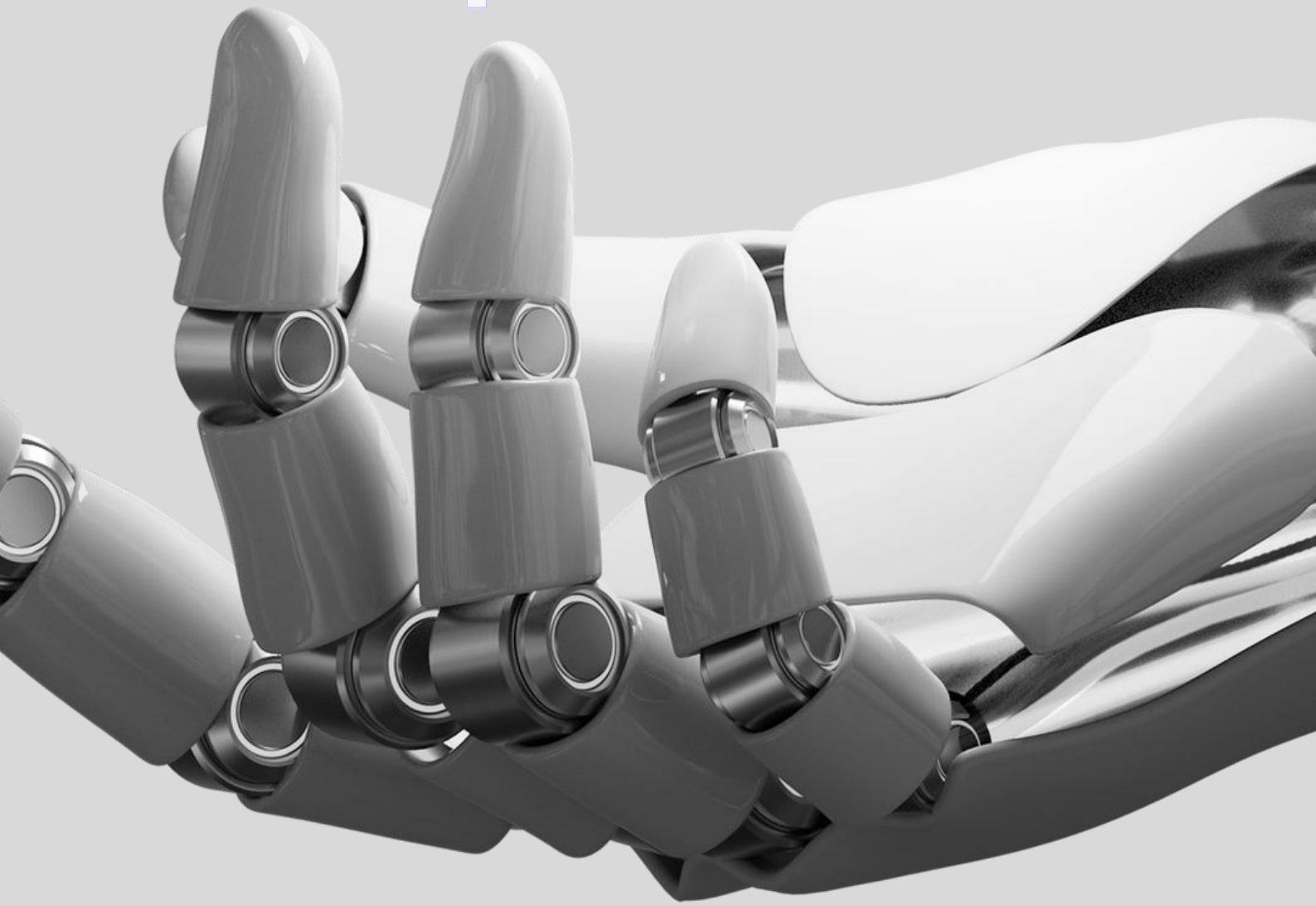


RL Final Report



Maria Aguilera García

1. What is Reinforcement Learning (RL)?

Reinforcement learning is at the crossroads of numerous scientific disciplines that are attempting to investigate the same topic of reinforcement learning, which is essentially the science of decision making. The discipline of engineering examines "Optimal Control," which seeks to comprehend the ideal way to make judgments and the optimal way to determine a series of actions to get the best results. Neuroscience investigates how humans make decisions and how a significant portion of the human brain is devoted to the dopamine system and, consequently, the reward system. In psychology, there is a great deal of work on classical and operational conditioning, which attempts to explain how animals make decisions – how and why animal behaviour occurs — for instance, if you give a dog a treat and it begins to salivate. There are three theories within economics: game theory, utility theory, and bounded rationality.

The founding principle of reinforcement learning is that optimal behaviour or activity is rewarded by a positive reinforcement. Similar to babies learning to walk, who modify their actions based on the results they encounter, such as taking smaller steps if the previous one caused them to fall. Using reinforcement learning techniques, machines and AI agents discover the optimal behaviour based on environmental input.

Therefore, Reinforcement Learning is a class of machine learning algorithms that help an autonomous agent navigate a complex environment. The agent must be given a sequence of rewards or penalties to learn what is required of it i.e., learn an optimal strategy for achieving a goal, by experimental trials. The agent attempts to maximize this reward over time or in mathematical terms to solve the bellman equation. The algorithms that help the agent estimate future reward fall under two classes: those that we require

2. How is RL different from Supervised or Unsupervised machine learning?

In reinforcement learning, we are not trying to predict an outcome, or predict the probability of something happening like in supervised learning for example. We are not even trying to cluster some events like we are doing in unsupervised learning. Instead, we are trying to teach a system to achieve a **goal**. We want a system to learn to do something.

In supervised learning, an input is given where the output of your model should produce is known. On the other side, in reinforcement learning, we have an input, and a network produces an output, but we don't know the target label. Us humans for example do not need thousands of examples to learn something; we learn by doing and this is the idea. The agent is learning as it interacts with the environment. There is no labelled data or pretrained data –we have **live** data. The agent uses live data to predict what strategy is best to achieve the given goal. He collects this data from the actions he takes, and he uses that to achieve the given goal. For example, the agent does not know if it should go up or down because we don't have a data set to train.¹ There is no

¹ In reinforcement learning, the network that transform input to output actions is called a Policy Network.

clear way to measure errors either like you would do in supervised learning where you could calculate this with accuracy for example.

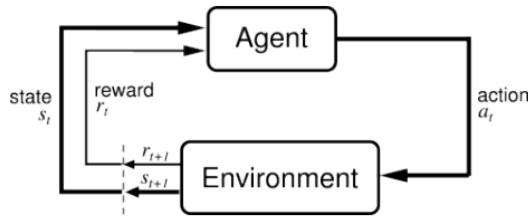
Therefore, the way in which reinforcement learning is different is that there is no supervisor; nobody instructs the agent on the proper behaviour. It is a paradigm of trial and error, with a simple a reward signal. The second distinction is that the reward signal does not arrive instantly; it may be delayed by several steps. The third contrast is that time is crucial in reinforcement learning; we are discussing sequential processes or stages. The data is not I.I.D data. Here, we have a dynamic system in which an agent traverses a universe in which what the agent observes in one setting correlates to what he sees in the next setting. Finally, the agent's activities determine the subsequent data it receives, i.e., the agent influences the data it receives.

3. What are Markov Decision Process (MDP) and Bellman's equation?

Reinforcement learning basically boils down to an **agent** interacting with an unknown **environment**, where this interaction gives **rewards** to the agent. The rewards measure how well the agent is doing and how we are defining the policy. From moving from one **state** to the other we have **Transition probabilities**, i.e. the problems will have some stochastic property. For example, when driving we do not know how likely we are we are going to find a car in the next street. Therefore, the state in which the agent will arrive is decided by the transition probabilities between states. The agent keeps track of these state transitions, actions, and rewards and looks for patterns to maximize its total reward over time. The algorithm that dictates how the agent will act in any given or state of the environment is called its policy. It is expressed as the probability of selecting a given action given the environment in some state. The agent's policy provides a guideline on what is the optimal action to take in a particular state, where each state is associated with a **Value Function**. A value function predicts the expected amount of future reward we can receive in this state, by following the corresponding policy.

In practice, what we get is a sequence of decisions by the agent and each decision does not just influence its immediate reward rather each decision influences all future reward. In mathematical terms, we have a sequence of states, actions, and rewards that one could call a decision process. If this state in this process is purely a function of the previous state and action of the agent, then this process is called a Markov Decision Process (MDP). These are an idealized mathematical abstraction that we use to construct the theory of reinforcement learning.

A Markov decision process has an agent, the agent gets to choose an action at time $t \rightarrow a_t$, as a consequence the environment changes which is the world around the agent (which could include the robot -the agent is the brain). There is a feedback cycle that the next situation you take, is a consequence of what you did before. An MDP is defined by:



Assumption: agent gets to observe the state

- Set of States S , in which a state is a status the agent (decision-maker) can hold.
- Set of actions A , in which an action is a movement (decision) agent can choose). These move the agent between states, with certain penalties or rewards.
- Transition function $P(s' | s, a)$, describes the probability of ending up in a state s' given an action a , of a given state.
- Reward function $R(s, a, s') \rightarrow$ reward function that says if you are in state s , took action a and landed at s' , how much reward you get for that.
- Start state s_0 or initial state distribution
- Discount factor gamma $\gamma \rightarrow$ how much you care about the future compared to current time
- Horizon $H \rightarrow$ how long you are going to be acting (sometimes infinity; sometimes shorter than that)

The Markov Property is crucial to highlight since it applies not just to Markov Decision Processes but to everything Markov-related (like a Markov Chain). It claims that the future state may be identified only by the present state, with no need for 'memory.'

The objective would be to maximize expected sum of rewards, under policy π , which is a prescription for each state what action you want to take

$$\max_{\pi} E[\sum_{t=0}^H \gamma^t (R_t, A_t, S_{t+1}) | \pi]$$

The mathematical relationship between the state transitions, rewards and actions and the policy is known as the Bellman Equation and it tells us the value meaning the expected future reward of a policy for some state of the environment. Reinforcement Learning sometimes (now always) means maximizing or solving the bellman equation. This desire to maximize future rewards leads to a dilemma—should the agent maximize its short-term reward by exploiting the best-known action or should it be adventurous and choose actions whose reward appears smaller or maybe even unknown?

Bellman Equation

The Bellman Equation is a fundamental concept in Markov Decision Processes. It describes a methodology for estimating the optimum anticipated reward at a given state by addressing the question, "What is the greatest benefit an agent may earn if they choose the optimal action now and in all future decisions?" it is the equation we use to find the value of each state and we solve it to find how good is that state for getting to an objective we are trying to achieve. The bellman equation breaks the value function into immediate reward plus discounted future values.

$$G(t) = R_{t+1} + \gamma R_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

Agents face a trade-off being should it take the action that gives the greatest immediate reward, or should it explore other actions to see if it can do better. This is called the explore exploit dilemma.

The discount factor γ plays an important role in determining the optimal reward. If it is set to 0, the model only cares about immediate reward. On the contrary, if it is set to 1, the model weights potential future reward just as much as it weights immediate rewards. Two other reasons is we use a discount factor is because a lot of algorithms thanks to the discount factor converge, and without it you might be able to get infinite rewards—with the discount factor the problems become bounded. The other reason is due to variance reduction.

4. What is a Dynamic Programming (DP) method and how is it different from MDP methods?

When we think about reinforcement learning, we are often thinking about the algorithm the agent uses to solve the Bellman Equation. These generally fall into two categories: algorithms that require a full model of their environment and algorithms that don't. As we mentioned earlier, actions caused the environment to transition from one state to another with some probability. Having a full model of the environment means knowing all the state transition probabilities with certainty. This means we require a complete accurate model of the environment. Of course, it's quite rare to have a complete accurate model of the environment and so the algorithms that require a full model of the environment are somehow of limited utility. These kinds of algorithms are known as Dynamic Programming. If we do not have a model or a model of the environment, we can't use dynamic programming. Instead, we have to rely on the family of model free algorithms e.g., Q-learning or deep Q learning. These rely on keeping track of the state transitions, actions, and rewards to learn the model on the environment over time. Therefore, Dynamic Programming is a group of algorithms that can solve a problem when we have a perfect model of the environment (i.e., the probability distributions of each change occurring in the problem setting are known) and an agent can only undertake discrete actions.

Therefore, dynamic programming is a model-based reinforcement learning family of algorithms. In the context of a Markov Decision Process, we assume that the environment that our agent is interacting with is a Markov Decision Process, where each state depends only on the previous state and the action taken by the agent.

Each time t , the agent has to decide an action to take A , to try to hopefully maximize the future reward. To do this we are trying to decide an optimal policy $\pi(s, a) = Pr(a = a | s = s)$. The value function $V_\pi(s) = E(\sum \gamma^t r^t | s_0 = s)$ that given our policy π , tells us what the expected future rewards is of being in this state now and acting this policy π which again is a probabilistic process because we are taking the "expected" value of our future rewards given some policy π . We don't know the best policy π nor the true value function for that best policy.

What we do assume in these models based algorithms is that we assume that we have an environment for how the environment works, how the system works. We assume the environment evolves according to a Markov Decision Process: $P(s' | s, a) = Pr(s_{k+1} = s' | s_k = s, a_k = a)$ so given a current state s and a current action a , there is some probability that my system will transition to a new state s' . We also assume that we have some model of what the reward structure looks like $R(s' | s, a) = Pr(r_{k+1} | s_{k+1} = s' | s_k = s, a_k = a)$, which is kind of like an assumption that you know the rules of the game you are playing. For example, if

you are playing chess, it is true the reward comes at the end of the game but you know how to checkmate someone, which is something you need to know to play.

Assuming that we knew the absolute best policy, the value function could be reformulated as $V(s) = \max_{\pi} E(\sum_{k=0}^{\infty} \gamma^k r_k | s_0 = s)$. Bellman reformulated this value function broke this into the current reward plus the sum of all rewards given that my next state s_1 is s' : $V(s) = \max_{\pi} E(r_0 + \sum_{k=0}^{\infty} \gamma^k r_k | s_1 = s')$ which can be rewritten as $V(s) = \max_{\pi} E(r_0 + \gamma(V(s')))$ and so you can recursively define your value function like this in which your value function in your current state is a function of the value function in the next state s' . this is Bellman's optimality condition. Once we have the optimal value function, we can extract the optimal policy $\pi = \operatorname{argmax}_{\pi} E(r_0 + \gamma(V(s')))$ as the policy π that maximizes this value function. So given systems that are recursive subproblems, Richard Bellman stated that given a multi-step optimization problem can be broken down into smaller sub problems.

Two of the leading algorithms that rely on dynamic programming for model -based reinforcement learning are value iteration and policy iteration.

In policy iteration, we pick a policy π at random to start. Then, we evaluate and improve the policy over and over again until convergence. At first, we don't care if the first policy is the best one or not. During execution, we focus on making it better with each iteration by repeating steps to evaluate the policy and make it better. Using this algorithm, we make a chain of policies, each of which is better than the one before it. We take steps to evaluate and improve policies until they can't get better anymore. The specified process is finite because a finite MDP has a limited number of policies. Ultimately, convergence between an optimum policy and an ideal value function is assured.

In value iteration, we compute the optimal state value function by iteratively updating the estimate $V(S)$. We start with a random value function $V(S)$ and at each step we update it. Hence, we look-ahead one step and go over all possible actions at each iteration to find the Optimal Value Function:

$$V^*(s) = \max_{\pi} E[\sum_{t=0}^H \gamma^t R(s_t, a_t, s_{t+1}) | \pi, s_0 = s]$$

Similar to the policy iteration algorithm's update step. The difference is that value iteration takes the maximum of all available actions. It changes the state value function in one step by calculating all possible future rewards.

5. What is a Multi-armed Bandit problem, and what are example applications of it in real-life?

Exploration vs. exploitation is a question that comes up in many parts of our lives. Say, your favourite restaurant is just around the corner. If you go there every day, you'll get used to what you get and miss out on the chance to find something even better. If you always go to new places, you'll probably have to eat some things you don't like. In the same way, online advisors try to find a balance between the ads that they know work well and new ads that might work even better. Another example would be when evaluating which stock will gives us the highest return.

If we knew everything about the environment, we wouldn't face this dilemma. Incomplete knowledge is what creates this dilemma: we need enough information to make good decisions while minimizing risks. Exploitation takes advantage of the best option at hand while exploration means taking risks to find new ways of doing things. Exploration means taking risks to find new ways of doing things. Long-term success may require short-term sacrifices. One exploratory try may fail, but it cautions us not to do so again. If we try something out and it doesn't work, we learn not to do it again.

One of the fundamental questions in reinforcement learning is how can an agent balance *exploration*—figuring out what is going on in the environment, with *exploitation* which means getting as much reward as possible along the way. It is this online decision-making that creates this dilemma. We are not given a batch of existing data like in supervised learning where we can go over that data set and make the one best decision. We are in the setting that we are gathering information as we go and the actions that we take affects the data that we see sometimes it worth to take new actions to get new data. Exploitation would be to take the best decision possible given the current information, while exploration would mean to gather more information which might lead us to make better decisions in the long run. Therefore, the best long-term strategy might actually involve giving up reward in the short term.

The name is derived from the image of a gambler at a casino with many slot machines, each of which is programmed with an unknown chance of receiving a pay-out for a single spin. In this scenario, you must decide which machines to play, how many times to play each machine and in what order to play them, and whether to stick with the current machine or try a different machine. The question is, "What is the most effective method for maximizing long-term returns?" In a situation with an endless number of trials, a naive strategy would be to continue playing with a single machine for countless rounds in order to estimate the "actual" payoff probability using the law of large numbers. This, however, is highly inefficient and does not ensure the greatest long-term benefit.

6. Describe the following bandits methods, and how these work in high level: Epsilon Greedy, Upper Confidence Bounds (UCB), Thompson Sampling.

Epsilon Greedy, Upper Confidence Bounds (UCB) and Thomson Sampling are solutions to maximize the quality of our decision in regard to the multi-armed bandit problem. To explain these, let's imagine we are in the situation

In ϵ -greedy algorithm we define an explicit threshold which determines when to exploit and when to explore. Going back to the casino example where we have 1,000 casino tokens, given that we choose an epsilon value of 0.1, and we begin inserting tokens and pulling arms. We could start randomly or depending on the nature of the problem, we may decide to take a very small baseline sample to get started. Using our epsilon value, we will make a choice before pulling any lever: 90% of the time we will put our token in the machine that up to this point has shown to have the highest average return but that other 10% of the time we will pull a random arm. Hopefully, we chose a small enough epsilon so that the majority of our tokens will be devoted to the best option but there will always be a small factor ensuring that we continue to explore

A different angle of approach blends together exploration and exploitation by considering how we will maximize our understanding of the rewards the machines are giving us. Up to this point, we have only considered the average payout for determining the expected value of a slot machine but tracking a simple average only gives us a simple glimpse into the nature of the reward distribution. For example, what about the variance of the rewards. One slot machine might yield a pay-out range of 0 to 10\$, while the second might yield a pay-out range of 4 to 6\$. Though these values will always remain unknown to us, we would eventually discover that both machines have an expected value of 5\$ by averaging the rewards over time. However, as we sample from an unknown distribution, we know that the average of the values we observe will get closer and closer to the average of that unknown distribution that we are sampling from. The idea of "closer and closer" means that as we collect these samples, we can also calculate a confidence interval for our estimate for that value. We can say our calculated average gets closer to the distributions' average because this confidence interval shrinks over time, the more samples we collect. In other words, the more times we explore a certain slot machine, the more certainty we gain over our estimate of its expected value. Therefore, due to the lower variance of the second machine's reward (4\$-6\$) we should be able to achieve confidence in our 5\$ expectation in far fewer trials than it would take to determine the same for the first machine (0\$-10\$). That means that tokens used for exploration in the second machine would be less valuable, as every time we use a token to explore, we are losing the opportunity of using that token to exploit. So the idea is that we should look to minimize exploration, especially when it is not actually given us useful information, and this is the concept that the Upper Confidence Bound (UCB) uses to address the exploration and exploitation dilemma. Instead of naively, taking the decision upfront of whether or not upfront, UCB keeps track of which options would benefit more or less from exploration and explores accordingly. Its' strategy is to take samples of each options' rewards, it considers the variance among the observed samples to also calculate an upper confidence bound. This upper confidence bound when added to the average represents the statistical potential for reward. Each round, the slot machine with the highest potential reward is chosen which provides a dual benefit. First, that is the option for which exploration would yield the most actionable newer information and second it is likely that that slot machine is already ranked highly among our winners, if not the top. With UCB, exploration and exploitation are achieved at the same time and principles of sampling statistics will dictate when option is due for further review as the confidence intervals of frequently chosen options shrink.

So far, we've talked about algorithms that use the average of previous trials to update the expected reward for each action. Thompson's algorithm for sampling does things in a different way. In this case, the bandit doesn't keep track of the actual payoff after each trial. Instead, it keeps beliefs about what it thinks the real payoff of each machine is. It works if you think like a Bayesian –It assumes that each action gives a reward based on a beta probability distribution and tries to estimate the distribution within a confidence interval. After an action is taken, the order in which the actions are done is changed. Then, the algorithm chooses the action that will give the best chance of getting the most reward. In this way, the Thompson sampling algorithm uses a method called "Bayesian Inference," in which the settings are changed as more evidence comes in.

7. What is the difference between Model-based and Model-free RL? Describe a method for each

A model simulates the behaviour of the environment or, more broadly, allows for assumptions about how the environment will act. Models are used for planning, which is any method of deciding on a course of action by examining potential future scenarios before they occur.

A model-based approach is when you have a way to express the environment i.e., it consists of a model of the environment. It involves an agent attempting to comprehend its surrounding and developing a model based on its interaction with them. In such a system, preference take precedence over the consequences of actions, i.e., the greedy agent will always strive to an action that will result in the great reward, regardless of the repercussions. This approach relies on planning as their primary component. Consider that the algorithm is model based if the agent can anticipate the reward for some action before actually completing it, thus predicting what it should do. The two approaches described in question 4 (policy iteration and value iteration) that fall in Dynamic Programming are examples of model-based approaches. These allow you to walk essentially iteratively through the Markov Decision Process iteratively and then assessing what the value of that action and state actually are and then refining and iterating the policy and value function.

On the other hand, if it is a model-free approach, the agent must do the action in order to see the results and learn from them. A model-free approach primarily relies on learning. The agent learns as it goes: “on the fly”. It seeks to learn the consequences of its actions through experience via algorithms such as Policy Gradient. In other words, such an algorithm will repeat an action and alter the policy (the method behind its actions) for optimum rewards depending on the results. It can be thought of as an “explicit” trial and error algorithm. Another method that falls under model-free approaches of the would be the Monte Carlo method which is a way of solving RL problems based on averaging sample returns, where you do not assume a complete knowledge of the environment, but rather require only experience. What this means is that instead of having a full representation of all possible states, actions and variations, this methos starts making some trial and making the outcome of these trials.

This leads in distinct applications for these two classes; for example, a model-based approach may be ideal for playing chess or for a robotic arm in a product assembly line, where the environment is static and getting the work done as efficiently as possible is our primary priority. However, in real-world applications such as self-driving automobiles, a model-based strategy may push the car to run over a pedestrian in order to get at its goal in less time (highest reward), while a model-free approach would have the car wait until the road is clear (optimal way out).

8. What is the difference between On-policy and Off-policy RL methods? Describe a method for each

Within model free methods in RL, there is on-policy (control) and off-policy (prediction). On policy learning means that the agent learns while interacting. The agent takes actions, and the policy is automatically updating while we are interacting with the system. We could say it is a method that “learns on the job” that attempt to evaluate or improve the policy that is used to make decisions. It is generally more stable, but less sample efficient. On the other hand, off policy

learning means learning over someone's shoulder, so it takes a series of iterations and uses all that knowledge to learn in one shot. This method is approach is generally less stable but more sample efficient.

A method that relies on the on-policy methods of RL is SARSA (state-action-reward-state-action) which is a method that allows us to adjust the Q function on the fly and avoids having mistaken due to exploration. It is the Q function which is the joint value of being in a particular state and taking a particular action. The off-policy version of SARSA is Q-learning which is basically based on the idea of adding random actions and in each episode, updating. In order to learn, it uses all the sample for updating the Q function.

9. What are the advantages and disadvantages of Monte Carlo (MC) methods compared to Temporal Difference (TD) methods?

Monte Carlo method is an offline method that's works by averaging sample returns. It doesn't require a full method of the environment like Dynamic Programming methods do. Instead, it involves letting an agent learn from the environment by interacting with it and collecting samples. One very important characteristic of this method is that it has to look at the full episode form the beginning to the end. It only works in episodic environments and these episodes must terminate. Therefore, it is not a very useful method when we have a continuous situation for example a machine that never ends. In this case, Monte Carlo is not a good approach as it only learns after a full episode.

Temporal difference (TD) is the best of Monte Carlo (MC) methods and Dynamic Programming (DP) methods. It will use sampling –in the sense that we don't need a complete knowledge of the environment (unlike DP methods) but it can learn from incomplete episodes, (unlike MC methods) meaning that we don't need to wait until the end to update our network. To decide when to stop it uses a hyperparameter called lambda (λ). Therefore, one of the advantages of TD over MC methods is that it can learn without knowing the final outcome, therefore even though you don't know whether it was a successful outcome or not it will still learn. Secondly, it can learn from incomplete sequences so you can decide when you want it to learn so that it is all the time learning and you don't have to wait. However, a trade-off of TD in comparison to MC methods it that it can have a lot of bias, depending on where you start the system and what are the initial values it will change the outcome very heavily. While Monte Carlo, is not so sensitive and doesn't have a lot of bias (pretty much 0 bias). Another disadvantage of TD vs MC is that it is less efficient than MC as it requires a lot of iterations while MC doesn't. However normally TD is more efficient than MC because again you are doing some sampling on incomplete data. It requires less computational power. The problem is that it doesn't converge all the time and can give you an error, as opposed to Monte Carlo that is more solid and almost converges all the time.

10. What is SARSA and how is it different from Q-Learning?

SARSA is an on-policy algorithm in which an action, A , is done in the current state, S , and the agent receives a reward, R . The agent then moves to the next state, S_1 , and performs action, A_1 in S_1 . As a result, the tuple (S, A, R, S_1, A_1) represents the abbreviation SARSA. It is referred to as an on-policy algorithm since it adjusts the policy in response to actions made. On the other hand, Q-learning on the other hand is an off-policy algorithm.

The difference between both is how the Q value (utility function) is updated after each action. SARSA chooses an action based on its current policy and updates its Q-values. Q learning on the other hand, selects the highest Q function from among all the feasible actions for the following step. Therefore while SARSA learns action values relative to the policy it follows, Q-learning does it relative to the greedy policy. They both converge to the real value function but at different rates. Q learnings tends to converge a bit slower and is not guaranteed to converge. Therefore SARSA is a better option when we care about the agent's performance during the process of training while Q-learning is better in situations where we don't care about the agent's performance during the training because what we care about it that it learns an optimal greedy policy.

11. If you have to choose between using Policy Gradient or Deep Q-Learning for the following 3 use cases, what will you use and why? An autonomous driving model based on RL, A chess playing bot based on RL, A stock market trading system based on RL.

Policy Gradient is a policy gradient method while Deep-Q-Learning is a value-based method. These are one of the major approaches used to solve RL problems, however there are some fundamental differences in the way they approach the action selection. While the purpose of Q-learning is to discover the greatest value in order to learn a single deterministic action from a discrete sequence of activities, the purpose of policy gradients and other direct policy searches is to discover a stochastic map from state to action that works in continuous action spaces. Also, when the Q function is too complex to be learned, DQN will fail. Furthermore, policy gradient methods usually converge faster but tend to converge to a local optimal. For this reason, policy-based method can solve problems that value based-methods cannot. Policy gradient methods can solve problems that have a large and continuous action space. Also, value-based methods cannot solve a problem where the optimal policy is stochastic, meaning that you can choose actions using a distribution for example Scissor/Paper/Rock. However, the advantage of value-based method over policy-based methods is their simplicity and their speed. A drawback of policy gradient methods is their high variance when estimating the gradient. Therefore, for an autonomous driving car and stock trading system, I would certainly employ a policy gradient method, and for playing chess I would employ a Deep Q-learning method.

12. Describe what is Deep Q-Learning and any important details about it.

Deep Q Learning is a value-based method, model free RL method that learns from actions that are outside the current policy. It uses the Q utility function but instead of using the Q-table that Q-learning method uses, it uses a Neural Network that takes a state and approximates the Q-values for each action based on that state. The advantage over Q-learning is that sometimes this Q-table is unfeasible for more complex games with hundreds of possible actions and states. Instead of updating the Q-table, the weights of the neural network are updated. Therefore, DQN is suitable for more complex games like Atari breakout game. Deep Q-learning consists of two neural networks: the local and target network, and the target network is updated after certain number of steps from the local network. It uses experience replay to learn from all past policies. It freezes target Q-network to avoid the moving target issue. Also, neural networks learn a representation from complex information, allowing DQN to leverage the computational superiority of deep neural networks to approximate the optimal Q value function. The most successful implementation of these algorithms are video games, robotic manipulation, and industry automation. The drawbacks is their overestimated value when taking the maximum over all actions as these Q values are very noisy. Another drawback is that their running time is very long and when an immediate response is needed in an example, other methods like policy gradient could be a better choice. Thirdly, they tend to overfit and not generalize well in other games as it wasn't trained that way. Lastly, is its lack of explainability due to it being a neural network.

13. Describe what is PPO and any important details about it.

Proximal Policy Optimization (PPO) is part of the policy gradient methods which at the same time are part of the policy-based method. It eliminates the drawback of most policy gradient methods of sample efficiency as they require to collect data to train and then the data is thrown away. Instead, PPO uses small updates to continue reusing the sample data and therefore eliminate the problem of sample efficiency. PPO enables multiple epochs of minibatch updates. Moreover, it alternates between sampling data through interactions with the environment and optimizing a surrogate objective function using stochastic gradient descent. Advantages: better overall performance, has the stability and reliability of trust-region methods (first order optimization), easy code implementation, simpler to implement than TRPO.

14. Describe what are A2C and A3C and any important details about it.

Actor critic methods combine the value based and policy approach, aiming to combine the advantages of both value-based and policy-based methods while eliminating all their drawbacks. They use deep convolutional Q learning as a base. The main distinction between A2C and A2B is the Asynchronous component. A3C is made up of numerous separate agents (networks) each with its own weights that interact with a distinct replica of the environment in concurrently. As a

result, they can cover a larger portion of the state-action space in considerably less time. You would use these kinds of methods when more accuracy is required.

15. In your opinion, what conditions should be met for considering using RL for solving a given problem?

When considering whether to use RL in order to solve a given problem, I would first evaluate whether another simpler method like supervised or unsupervised method is capable of solving it. In the case where it is not I would compare the different learning methods and see which one is the best suited for the problem. The criteria to choose a specific algorithm are many:

- 1) Sample efficiency- how many samples are needed to train a good policy?
- 2) Stability and convergence- how easily and quickly the model converges
- 3) Will the model generalize to other tasks
- 4) Assumptions and approximation- are there any restrictions to this method; is this approach applicable to discrete or continuous space?
- 5) Exploration- how thoroughly does it investigate the action space?
- 6) Model centric vs policy centric
- 7) Policy gradient vs Value learning
- 8) What are the costs? Real world stimulation is costly.
- 9) Bias vs variance